

PROYECTO FINAL

# INTÉRPRETE DE UN LENGUAJE IMPERATIVO

Fundamentos de Compiladores e Intérpretes

---

Arestigue Mamani Erickson Gabriel

Jauregui Peña Bruno Marcelo

Lavado Quispe Alessandro Jaren

UNIVERSIDAD NACIONAL DE INGENIERÍA



# | Objetivo del Proyecto

Construir un intérprete capaz de ejecutar un **mini-lenguaje imperativo** inspirado en Python, facilitando la ejecución directa de scripts.

## Soporte del Intérprete

</> Variables

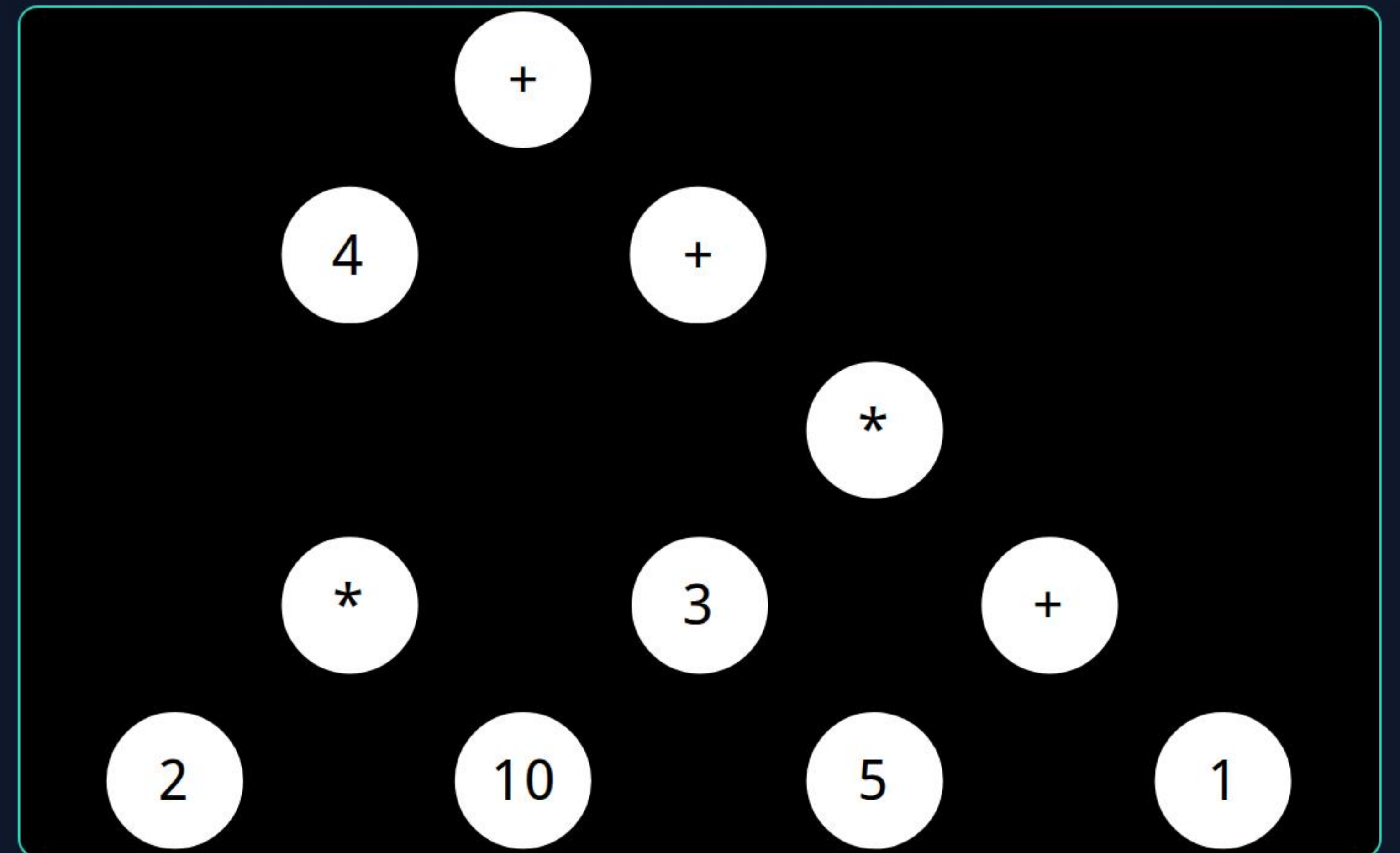
✓ Comparación

↺ If / While

📊 Aritmética

>\_ Print

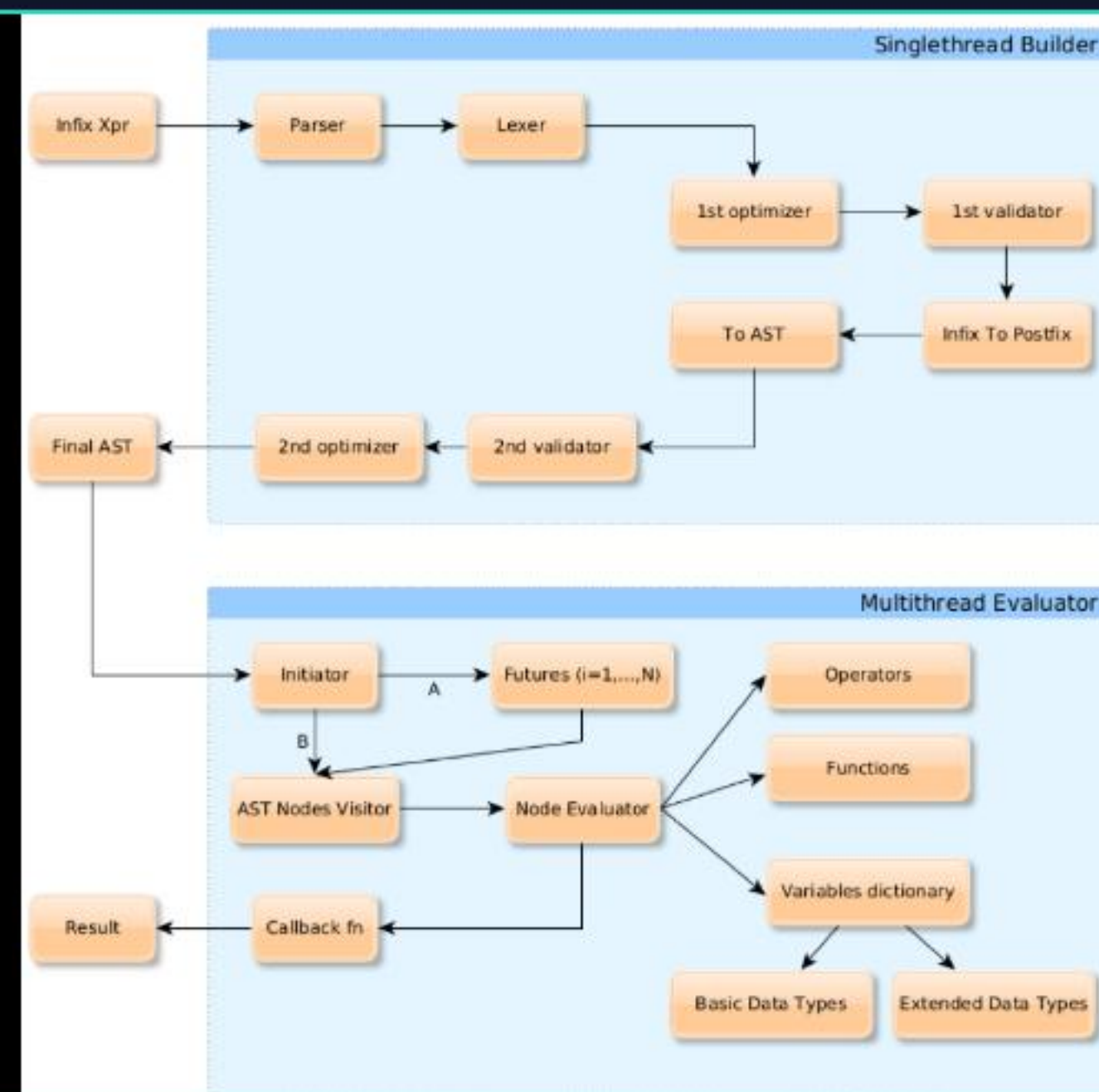
≡ For





# | ¿Qué es un Intérprete?

A diferencia de un compilador que genera un ejecutable previo, un intérprete **ejecuta directamente** el código fuente a través de un pipeline de procesamiento.

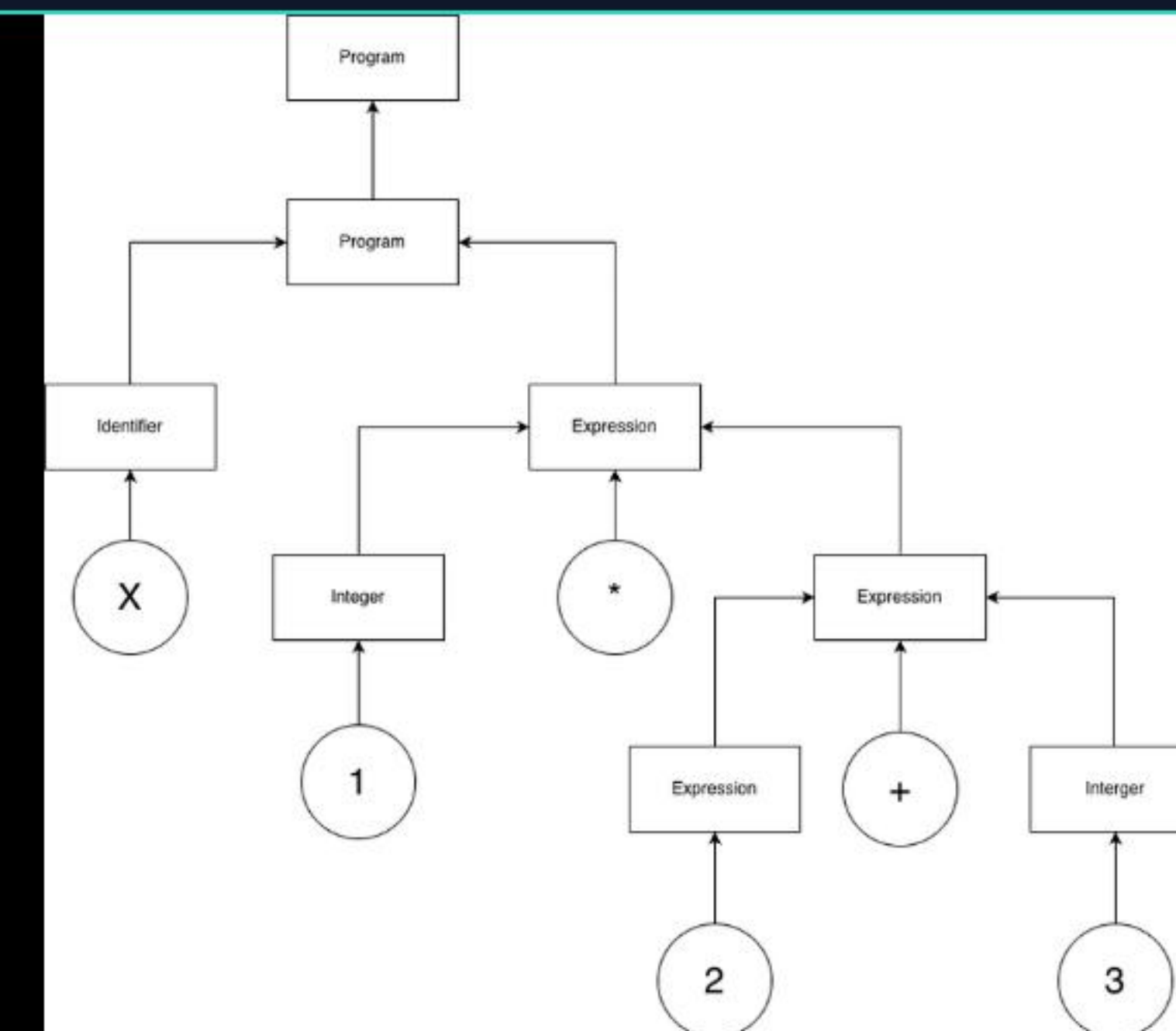




# Arquitectura del Proyecto

```
main.c (Coordinador)
  ↓
parse() → lexer.c / parser.c
  ↓
AST (Representación Intermedia)
  ↓
evaluate() → eval.c
  ↓
stdout (Salida Estándar)
```

El **main.c** no implementa lógica pesada; su función es **orquestrar el flujo** de datos desde el archivo fuente hasta la consola.





# | Análisis Léxico (Lexer)

El componente `lexer.c` identifica tokens usando funciones de `ctype.h` para validar tipos de caracteres.

Funciones Clave:

`getNextToken()`

Utiliza: `isdigit()`, `isalpha()`, `strcmp()`.

| Lexema | Token     |
|--------|-----------|
| x      | ID        |
| =      | ASSIGN    |
| 5      | NUM       |
| +      | PLUS      |
| 8      | NUM       |
| ;      | SEMICOLON |



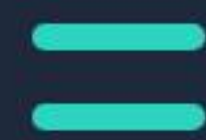
# | Tokens del Lenguaje



## Valores

`NUM, ID`

`+, -, *, /, %`



## Lógica

`=, ==, !=`

`<, >, <=, >=`



## Control

`print, if, else`

`while, for`

## Simbolos

`{ }, ( ), ;`



# | Parser Recursivo Descendente

El parser valida la estructura sintáctica siguiendo una jerarquía estricta que define la **precedencia matemática**.

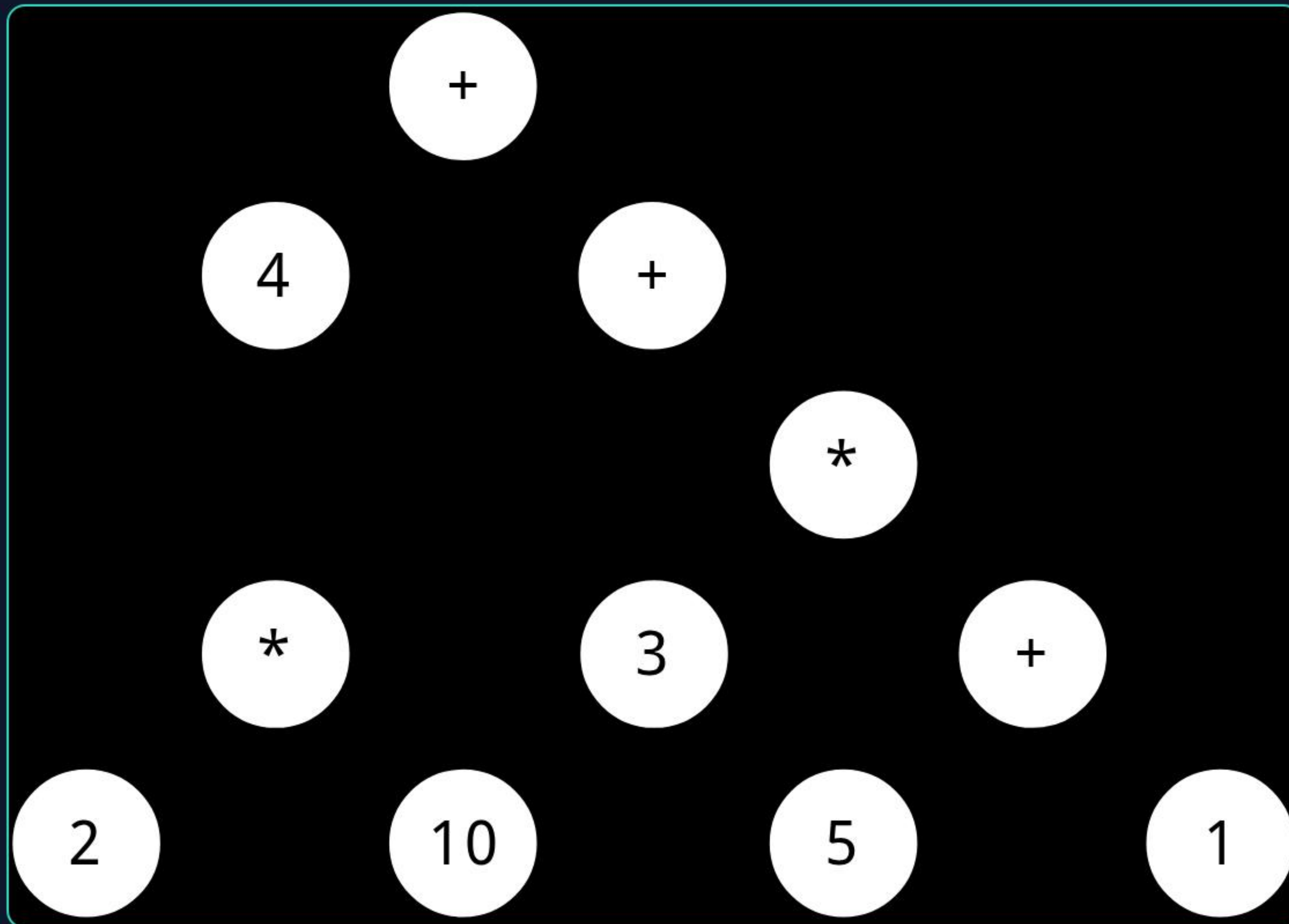
- ✓ `program()` → `statement()`
- ✓ `comparison()` → `expr()`
- ✓ `term()` → `factor()`

## Jerarquía Lógica

Esta estructura asegura que `3+5*2` se evalúe como `3+(5*2)` automáticamente.



# | Construcción del AST



El **Abstract Syntax Tree (AST)** representa la operación de forma jerárquica.

⌚ No se ejecuta inmediatamente.

🏗️ Se construye el árbol completo primero.

🔧 El evaluador recorre los nodos después.



# Definición de AST.H

```
typedef struct AST {  
    NodeType type;  
    int value; // NUM  
    char name[64]; // ID  
    char op[4]; // + - * /  
    struct AST *left, *right;  
    struct AST *third, *fourth;  
    struct AST *next;  
} AST;
```

## Punteros de navegación:

- ▶ **left/right**: Operandos.
- ▶ **third/fourth**: Condicionales/Bucles.
- ▶ **next**: Siguiete instrucción.



# | Tipos de Nodos (NodeType)

NODE\_NUM  
NODE\_VAR

NODE\_ASSIGN  
NODE\_PRINT

NODE\_IF  
NODE\_WHILE

NODE\_FOR  
NODE\_COMPARE

NODE\_BINOP  
NODE\_BLOCK

Código Fuente: **x = 10;**

| Produce un **NODE\_ASSIGN** con un hijo **NODE\_NUM**.



# | El Evaluador Lógico

## `evaluate(AST *node)`

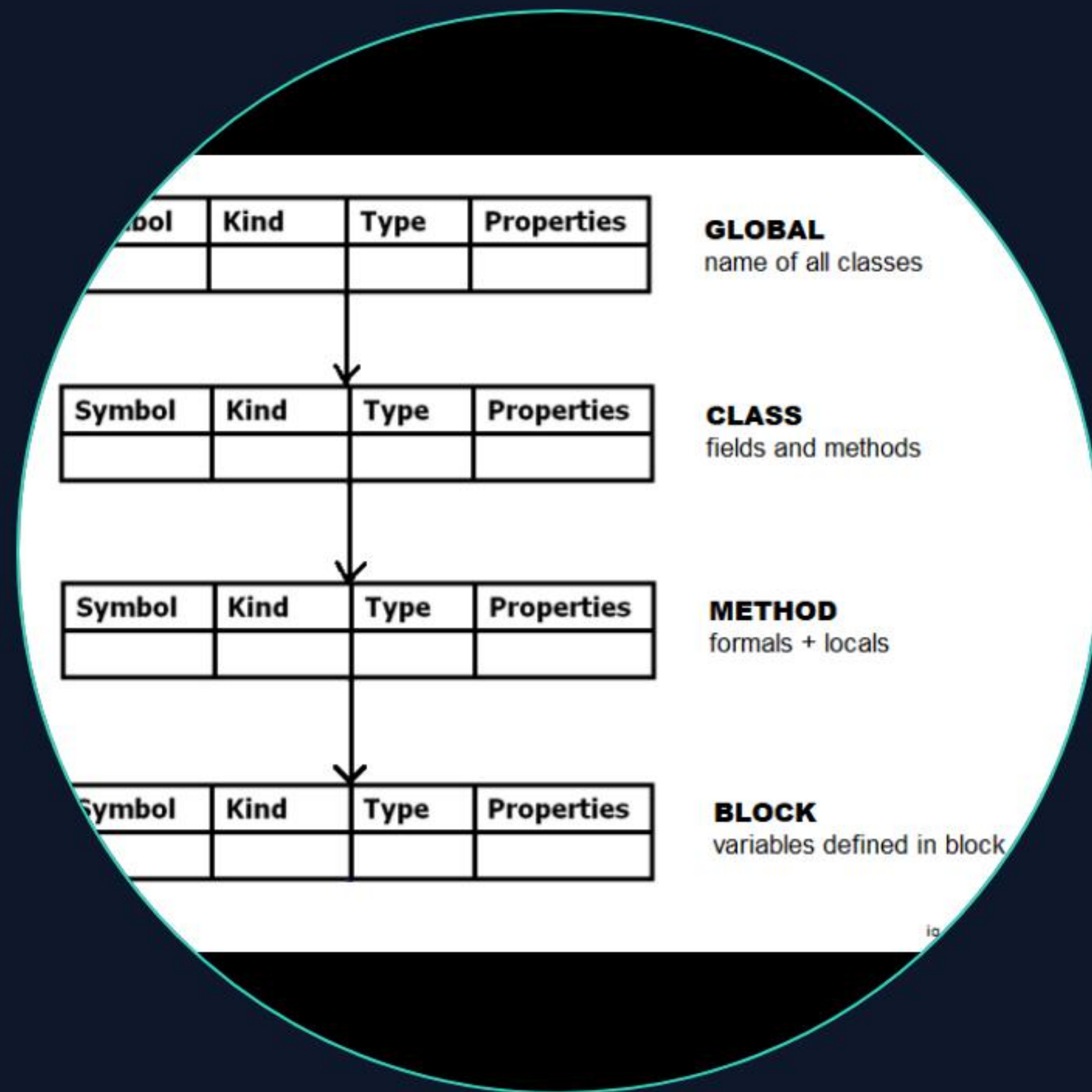
Es una función recursiva que procesa el árbol de abajo hacia arriba.

Utiliza una estructura `switch(node->type)` para decidir la ejecución.

```
switch(node->type) {  
    case NODE_NUM:  
        return node->value;  
    case NODE_BINOP:  
        return calc(node->op, node->left, node->right);  
}
```



# | Tabla de Símbolos



Permite la persistencia de datos durante la ejecución del programa.

**Symbol:** { char name[64]; int value; }

x → 15   y → 30   z → 100

Funciones: **setVar()** / **getVar()**



# | Control de Flujo

## Lógica de Ejecución

- 🔗 **IF:** Evalúa condición; si es  $\neq 0$ , ejecuta bloque.
- 🔄 **WHILE:** Repite bloque mientras condición sea verdadera.

```
if(a < 5)
```



Evalúa condición → **Verdadero** → Ejecuta bloque



# | Manejo de Errores



## Léxicos

Tokens desconocidos: @, #,  
\$



## Sintácticos

Estructura incompleta:  
print(5



## Semánticos

Errores lógicos en runtime: 10  
/ 0

```
> error: division por cero
```



# | Ejemplo Completo de Ejecución

```
x=5; y=10;  
print(x+y);  
if(x < y){  
    print(y);  
}
```

Código → Tokens → AST → Salida

**Salida: 15, 10**



# | Ventajas y Extensibilidad



## Diseño Modular

Separación clara entre lexer, parser, ast y eval.



## Extensibilidad

Fácil adición de funciones, booleanos y arrays.

**GRACIAS**  
UNI - 2024